

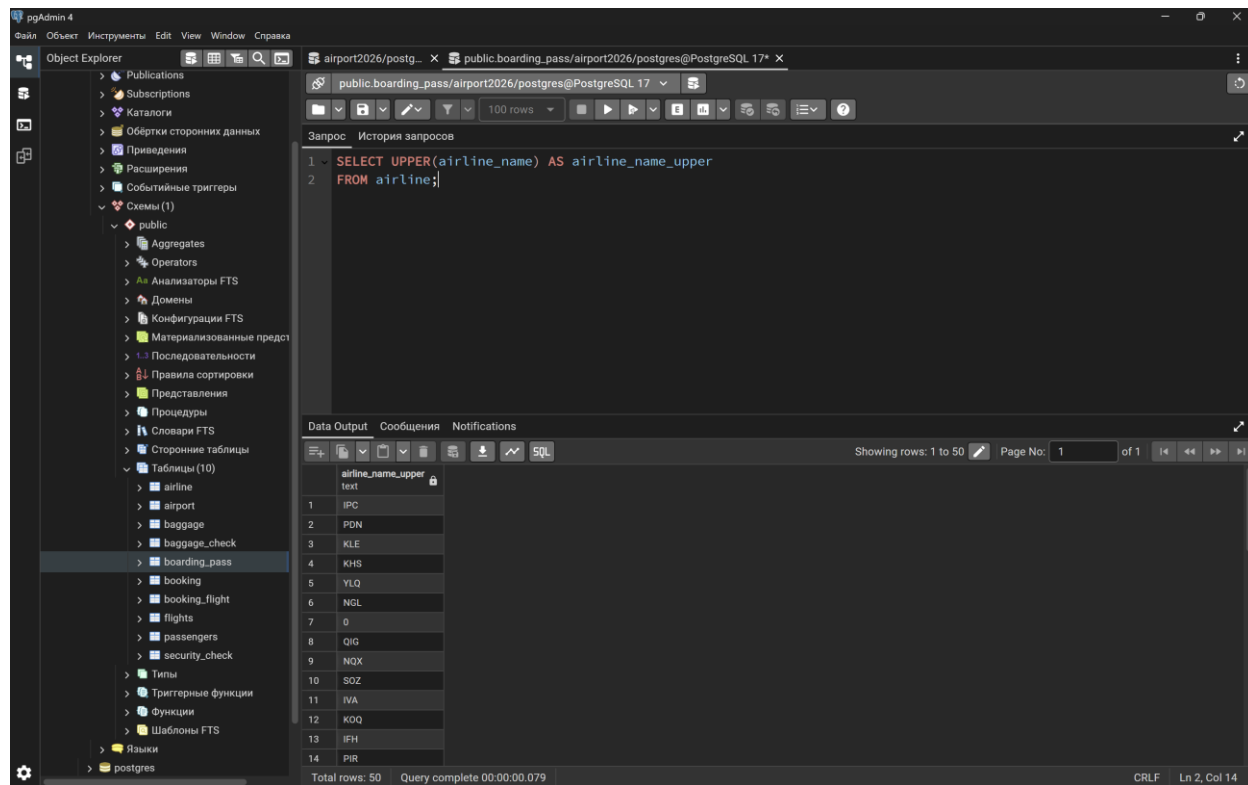
Laboratory work 4

We continue to work with the database from the previous laboratory works.

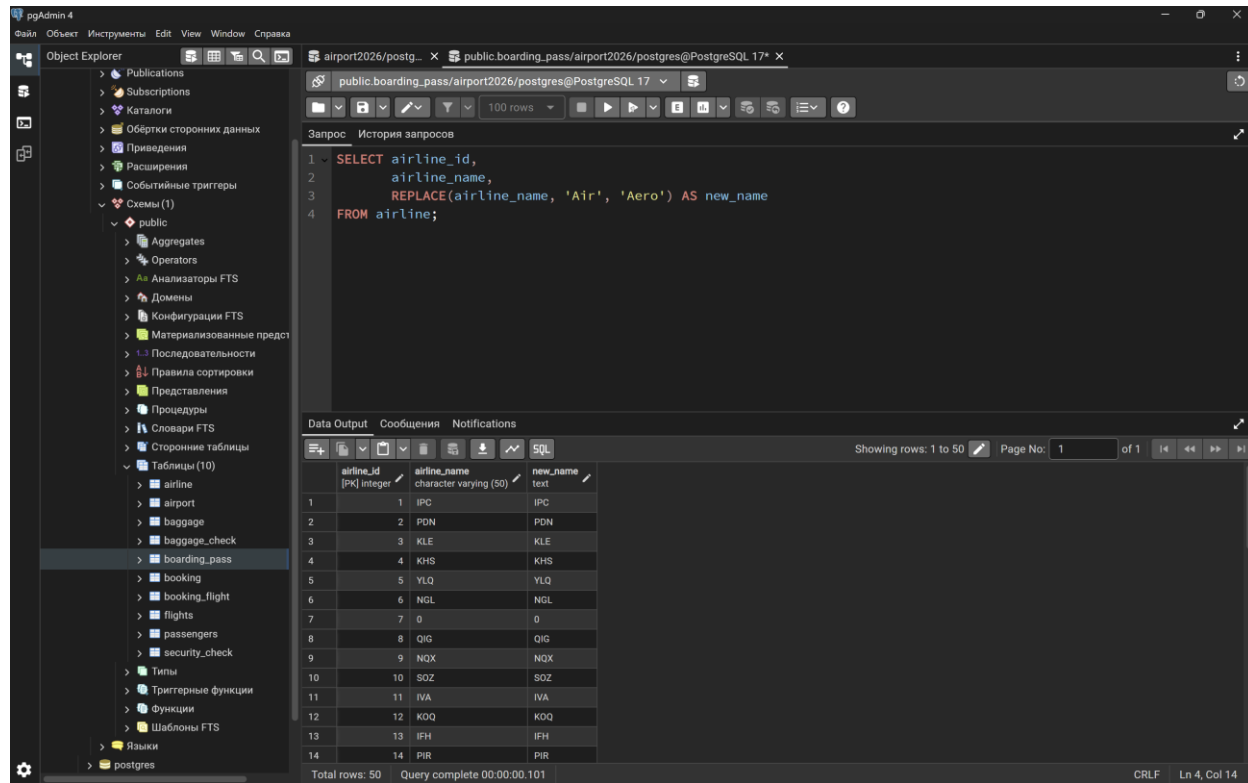
Take a full-page screenshot that covers the code and results of each task.

Tasks:

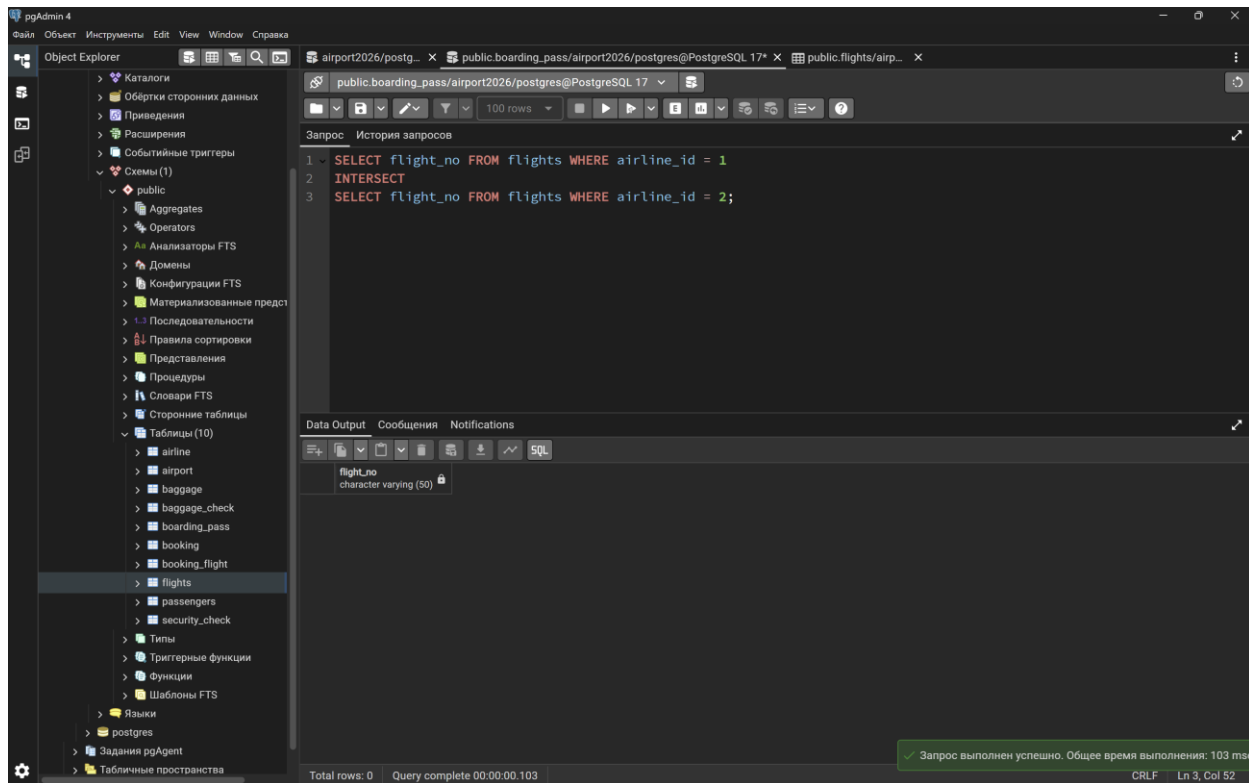
1. Retrieve all airline names in uppercase.



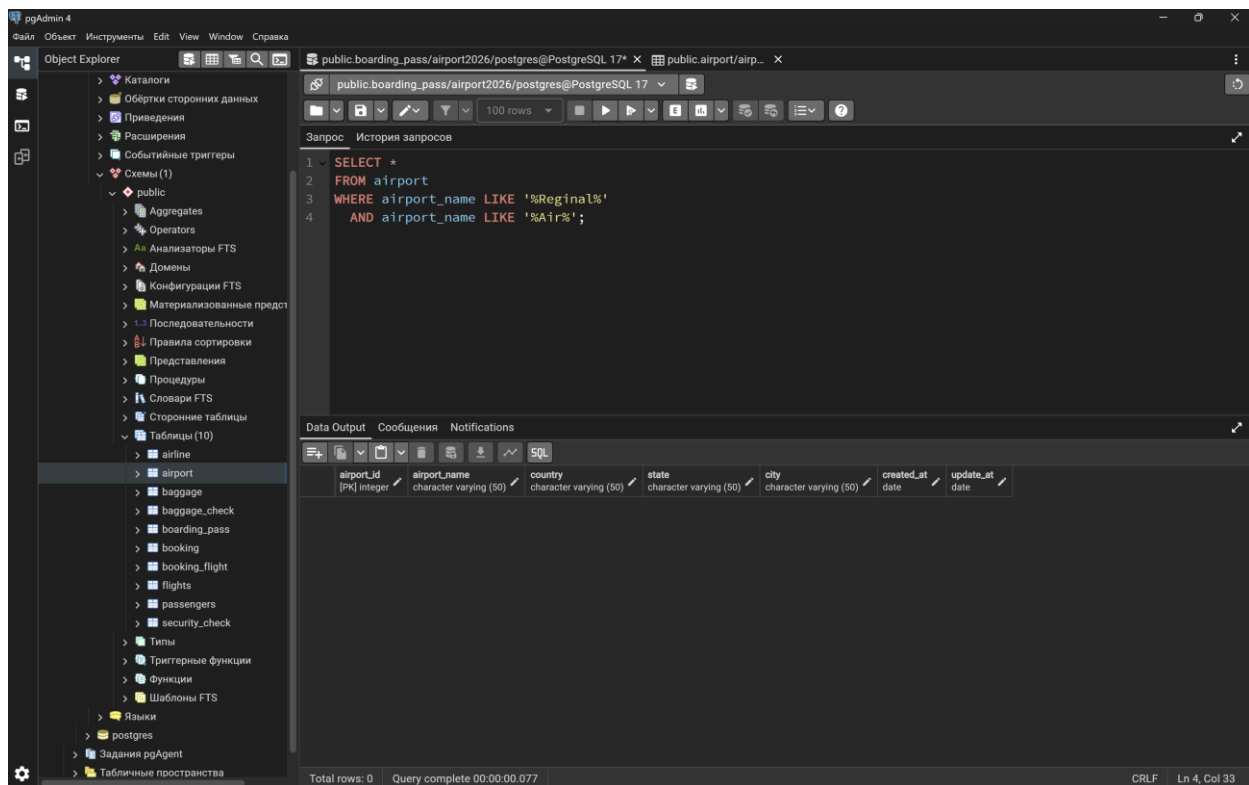
2. Replace any occurrence of the word "Air" in airline names with "Aero".



3. Find all flight numbers that coordinates with both airline 1 and airline



4. Retrieve airports that contain the word "Reginal" and "Air" in their names.



5. Retrieve passenger names and format their birth dates as 'Month DD, YYYY'.

pgAdmin 4

Object Explorer

- Каталоги
- Обертки сторонних данных
- Приведения
- Расширения
- Событийные триггеры
- Схемы (1)
 - public
 - Aggregates
 - Operators
 - Анализаторы FTS
 - Домены
 - Конфигурации FTS
 - Материализованные предст
 - Последовательности
 - Правила сортировки
 - Представления
 - Процедуры
 - Словари FTS
 - Сторонние таблицы
 - Таблицы (10)
 - airline
 - airport
 - baggage
 - baggage_check
 - boarding_pass
 - booking
 - booking_flight
 - flights
 - passengers
 - security_check
 - Типы
 - Триггерные функции
 - функции
 - Шаблоны FTS
 - Языки
 - postgres
 - Задания pgAgent
 - Табличные пространства

public.boarding_pass/airport2026/postgres@PostgreSQL 17* X public.airport/airp...

public.boarding_pass/airport2026/postgres@PostgreSQL 17

Запрос История запросов

```
1 SELECT first_name || ' ' || last_name AS full_name,  
2       TO_CHAR(date_of_birth, 'Month') || TO_CHAR(date_of_birth, 'DD, YYYY') AS birth_date  
3 FROM passengers;
```

Data Output Сообщения Notifications

Showing rows: 1 to 200 Page No: 1 of 1

	full_name text	birth_date text
1	Hilde Imis	January 03, 2000
2	Arvy Sparsholt	June 09, 1974
3	Reinald Pococke	June 07, 1982
4	Con Borrel	October 17, 1986
5	Wayne Bangs	April 22, 1996
6	Tildy Shackleford	April 15, 2004
7	Bytle Dram	July 07, 1985
8	Howard Igo	March 29, 1984
9	Alphonso Philippou	August 30, 1975
10	Fairfax Needham	July 08, 1981
11	Vanda Prime	May 17, 1977
12	Terra Wimmer	January 02, 1977
13	Domenic Blaszcak	December 31, 2001
14	Conrad Southerell	January 25, 1998

Total rows: 200 Query complete 00:00:00.070 CRLF Ln 3, Col 17

6. Find flight numbers that have been delayed based on the actual arrival time.

pgAdmin 4

Object Explorer

- Каталоги
- Обертки сторонних данных
- Приведения
- Расширения
- Событийные триггеры
- Схемы (1)
 - public
 - Aggregates
 - Operators
 - Анализаторы FTS
 - Домены
 - Конфигурации FTS
 - Материализованные предст
 - Последовательности
 - Правила сортировки
 - Представления
 - Процедуры
 - Словари FTS
 - Сторонние таблицы
 - Таблицы (10)
 - airline
 - airport
 - baggage
 - baggage_check
 - boarding_pass
 - booking
 - booking_flight
 - flights
 - passengers
 - security_check
 - Типы
 - Триггерные функции
 - функции
 - Шаблоны FTS
 - Языки
 - postgres
 - Задания pgAgent
 - Табличные пространства

public.boarding_pass/airport2026/postgres@PostgreSQL 17* X public.airport/airp...

public.boarding_pass/airport2026/postgres@PostgreSQL 17

Запрос История запросов

```
1 SELECT DISTINCT flight_no  
2 FROM flights  
3 WHERE actual_arrival > scheduled_arrival;
```

Data Output Сообщения Notifications

Showing rows: 1 to 324 Page No: 1 of 1

	flight_no character varying (50)
1	MY-12
2	KI-G
3	GA-6
4	NZ-AUK
5	BJ-DO
6	IN-MP
7	TH-32
8	FR-O
9	DE-BW
10	RU-VOR
11	CA-NU
12	ZA-NL
13	AF-FRA
14	AR-P

Total rows: 324 Query complete 00:00:00.097 CRLF Ln 3, Col 42

7. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time.

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with the 'public' schema selected. The central pane shows a SQL query:

```
SELECT flight_id, flight_no, scheduled_arrival, actual_arrival, (actual_arrival - scheduled_arrival) AS days_late FROM flights WHERE actual_arrival > scheduled_arrival ORDER BY days_late DESC;
```

 The right pane shows the 'Data Output' tab with a table of 14 rows. The table has columns: flight_id, flight_no, scheduled_arrival, actual_arrival, and days_late. The data shows flights that arrived late, with the largest delay being 345 days for flight 50.

flight_id	flight_no	scheduled_arrival	actual_arrival	days_late
50	AO-CCU	2023-04-07	2024-03-17	345
524	SB-RB	2023-04-01	2024-03-07	341
282	PG-MPL	2023-04-04	2024-03-02	333
730	US-AK	2023-04-08	2024-03-05	332
881	DZ-33	2023-04-19	2024-03-14	330
295	US-AK	2023-04-19	2024-03-11	327
620	PG-SAN	2023-03-22	2024-02-11	326
286	PA-7	2023-04-19	2024-03-03	319
962	SD-25	2023-04-05	2024-02-10	311
470	ID-JI	2023-04-14	2024-02-19	311
820	DE-TH	2023-04-27	2024-03-02	310
217	BF-TAP	2023-03-23	2024-01-27	310
327	US-UT	2023-04-22	2024-02-21	305
900	PF-U-A	2023-04-01	2024-01-29	303

8. Show airlines from any of: ('France','Portugal','Poland') created between '2023-11-01' and '2024-03-31'.

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with the 'public' schema selected. The central pane shows a SQL query:

```
SELECT * FROM airline WHERE airline_country IN ('France', 'Portugal', 'Poland') AND created_at BETWEEN '2023-11-01' AND '2024-03-31';
```

 The right pane shows the 'Data Output' tab with a table of 7 rows. The table has columns: airline_id, airline_code, airline_name, airline_country, created_at, and update_at. The data shows airlines from France, Portugal, and Poland created between November 1, 2023, and March 31, 2024.

airline_id	airline_code	airline_name	airline_country	created_at	update_at
4	OOKB	KHS	France	2023-11-18	2024-02-18
13	OIFE	IFH	Portugal	2024-01-09	2023-11-09
15	SYKZ	KKG	Poland	2023-12-24	2023-10-30
24	ZSRG	RUG	Portugal	2024-03-11	2023-08-24
28	RKPD	JDG	Poland	2023-11-03	2023-12-06
30	CYST	YST	Poland	2023-11-24	2023-10-07
43	AYKM	KMA	France	2023-11-21	2023-05-30

9. Find top3 overweighted baggage with more than 25kg.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'public' schema and the 'airport' database. The main pane shows a SQL query executed against the 'public.boarding_pass/airport2026/postgres@PostgreSQL 17' database. The query is as follows:

```
1 SELECT baggage_id, booking_id, weight_in_kg
2 FROM baggage
3 WHERE weight_in_kg > 25
4 ORDER BY weight_in_kg DESC
5 LIMIT 3;
```

The 'Data Output' pane displays the results of the query, showing the top 3 overweighted baggage items:

baggage_id [PK] integer	booking_id integer	weight_in_kg numeric (4,2)
1	321	349
2	61	464
3	102	53

The status bar at the bottom indicates: 'Total rows: 3 Query complete 00:00:00.131'. A green message box at the bottom right states: 'Запрос выполнен успешно. Общее время выполнения: 131 мсек. обработано строк: 3.'

10. Find the youngest passengers' full name.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'public' schema and the 'airport' database. The main pane shows a SQL query executed against the 'public.boarding_pass/airport2026/postgres@PostgreSQL 17' database. The query is as follows:

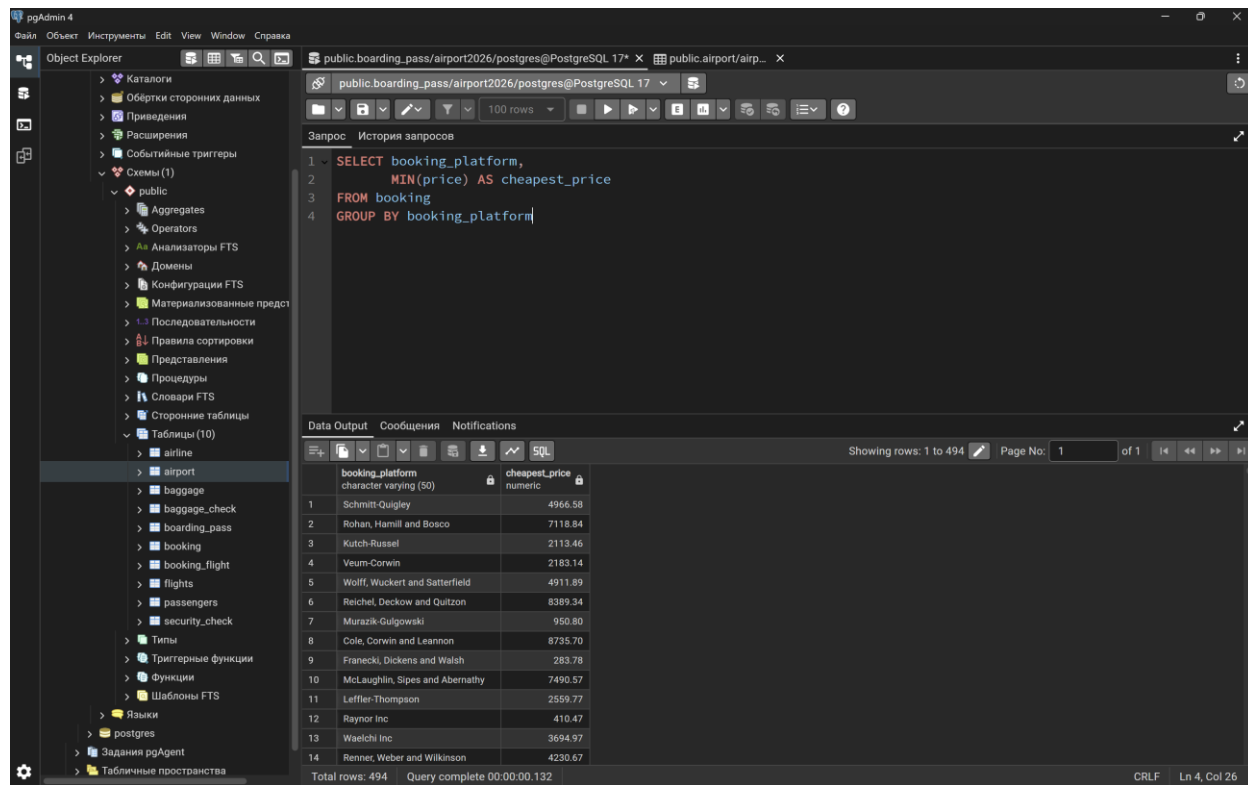
```
1 SELECT first_name || ' ' || last_name AS full_name,
2        date_of_birth
3 FROM passengers
4 ORDER BY date_of_birth DESC
5 Limit 1;
```

The 'Data Output' pane displays the results of the query, showing the youngest passenger's full name and date of birth:

full_name text	date_of_birth date
Bernie Michal	2009-11-03

The status bar at the bottom indicates: 'Total rows: 1 Query complete 00:00:00.061'. The CRLF status is shown as 'Ln 5, Col 9'.

11. Find the cheapest booking price on each booking platform.



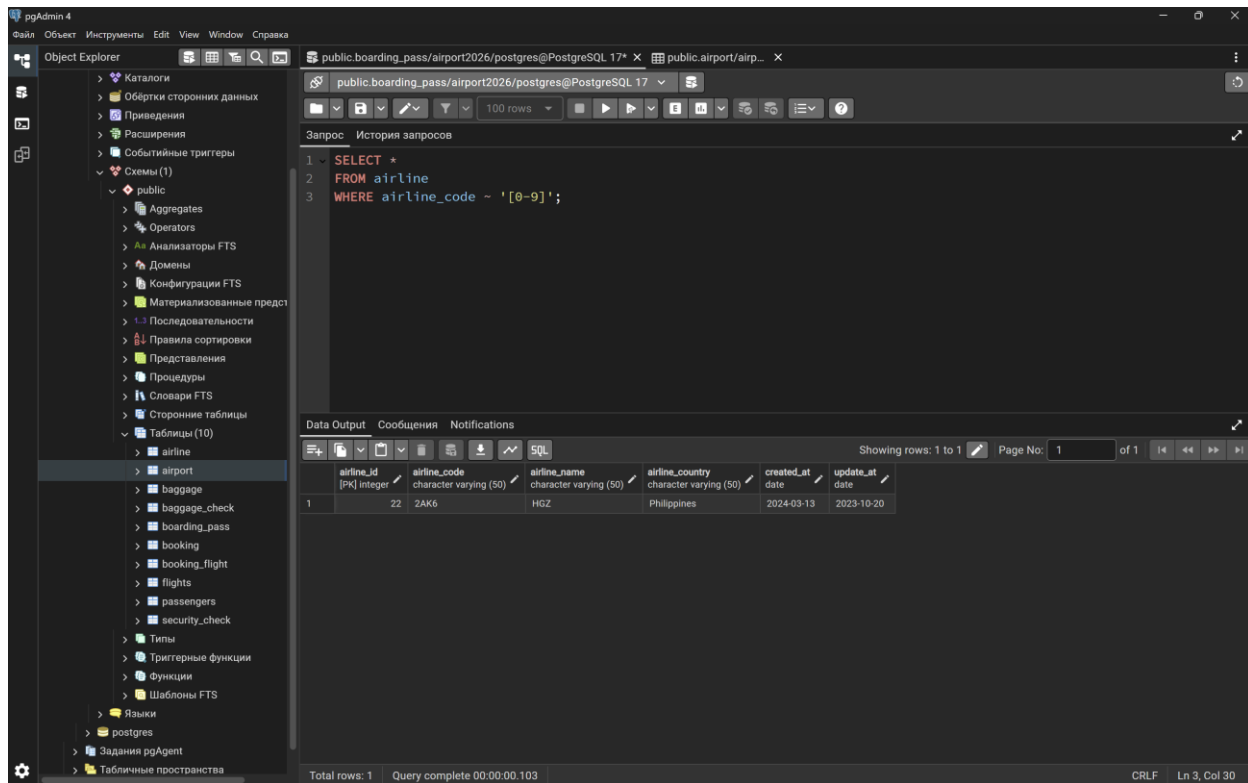
The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'public' schema and the 'airport' table. The central pane shows a SQL query:

```
1 SELECT booking_platform,  
2      MIN(price) AS cheapest_price  
3 FROM booking  
4 GROUP BY booking_platform
```

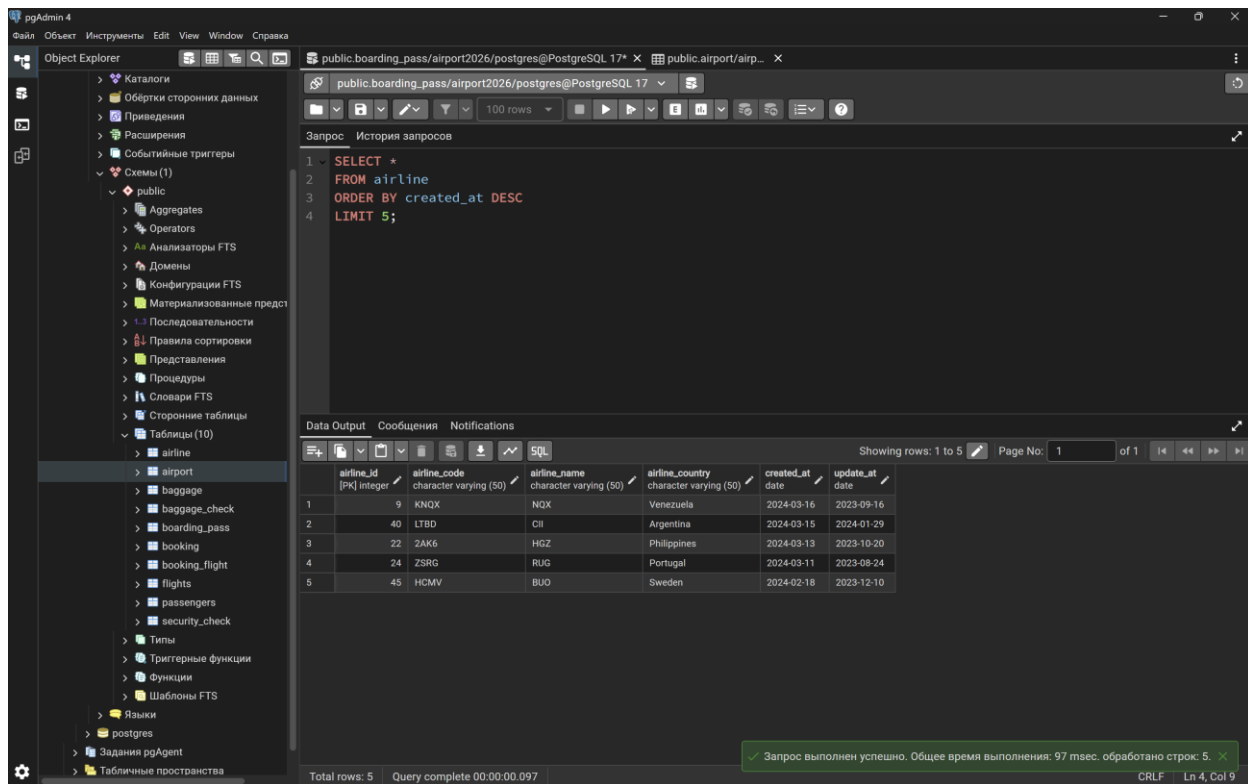
The bottom pane displays the query results in a table format. The table has two columns: 'booking_platform' (character varying (50)) and 'cheapest_price' (numeric). The results show 14 rows of data, with the first row being 'Schmitt-Quigley' at a price of 4966.58. The total number of rows is 494.

booking_platform	cheapest_price
Schmitt-Quigley	4966.58
Rohan, Hamill and Bosco	7118.84
Kutch-Russel	2113.46
Veum-Corwin	2183.14
Wolff, Wuckert and Satterfield	4911.89
Reichel, Deckow and Quitzon	8389.34
Murazik-Gulgowski	950.80
Cole, Corwin and Leannon	8735.70
Franecki, Dickens and Walsh	283.78
McLaughlin, Sipes and Abernathy	7490.57
Leffler-Thompson	2559.77
Raynor Inc	410.47
Waelchi Inc	3694.97
Renner, Weber and Wilkinson	4230.67

12. Return airlines whose airline_code contains a digit.



13. List the top5 most recently created airlines.



14. Return all rows where booking_id is between 200 and 300 inclusive and check_result <> 'Checked'.

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'public' schema and the 'baggage_check' table. The central pane shows the following SQL query:

```
1. SELECT *
2. FROM baggage_check
3. WHERE booking_id BETWEEN 200 AND 300
4. AND check_result <> 'Checked';
```

The bottom pane displays the query results in a table format. The table has 14 rows and 7 columns: baggage_check_id, check_result, created_at, update_at, booking_id, and passenger_id. The results show 14 rows of data where the booking_id is between 200 and 300 and the check_result is not 'Checked'.

baggage_check_id	check_result	created_at	update_at	booking_id	passenger_id
1	Not checked	2023-05-04	2023-04-29	226	44
2	Not checked	2023-10-02	2023-06-22	288	133
3	Not checked	2023-06-14	2023-11-10	246	101
4	Not checked	2023-05-29	2023-06-02	284	22
5	Not checked	2024-02-17	2023-10-03	267	68
6	Not checked	2024-01-20	2023-06-10	237	140
7	Not checked	2023-06-12	2023-11-05	279	195
8	Not checked	2023-12-02	2023-05-28	274	187
9	Not checked	2023-11-03	2023-12-03	300	130
10	Not checked	2023-10-04	2023-04-26	259	63
11	Not checked	2023-05-21	2023-09-03	234	76
12	Not checked	2024-01-26	2023-04-07	260	154
13	Not checked	2024-03-06	2023-05-01	218	188
14	Not checked	2023-07-16	2024-02-09	256	109

15. Baggage checks where update_at is in the same month as created_at but occurs earlier than created_at.

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'public' schema and the 'baggage_check' table. The central pane shows the following SQL query:

```
1. SELECT *
2. from baggage_check
3. where extract(month from update_at) = extract(month from created_at) and update_at < created_at
```

The bottom pane displays the query results in a table format. The table has 14 rows and 7 columns: baggage_check_id, check_result, created_at, update_at, booking_id, and passenger_id. The results show 14 rows of data where the update_at is in the same month as created_at and occurs earlier than created_at.

baggage_check_id	check_result	created_at	update_at	booking_id	passenger_id
1	34	2023-04-25	2023-04-08	28	57
2	86	2023-12-25	2023-12-07	312	22
3	104	2024-03-17	2024-03-14	41	83
4	109	2023-08-30	2023-08-09	469	91
5	119	2023-12-02	2023-12-01	135	55
6	148	2024-03-17	2024-03-16	86	5
7	219	2023-04-17	2023-04-14	233	200
8	248	2023-06-22	2023-06-09	449	60
9	251	2023-06-19	2023-06-16	260	26
10	299	2023-04-26	2023-04-08	385	191
11	304	2023-07-29	2023-07-22	210	10
12	314	2024-03-07	2023-03-28	254	57
13	396	2023-07-13	2023-07-01	37	144
14	404	2024-03-04	2023-03-29	253	121